

Job Interview in the AI-Era

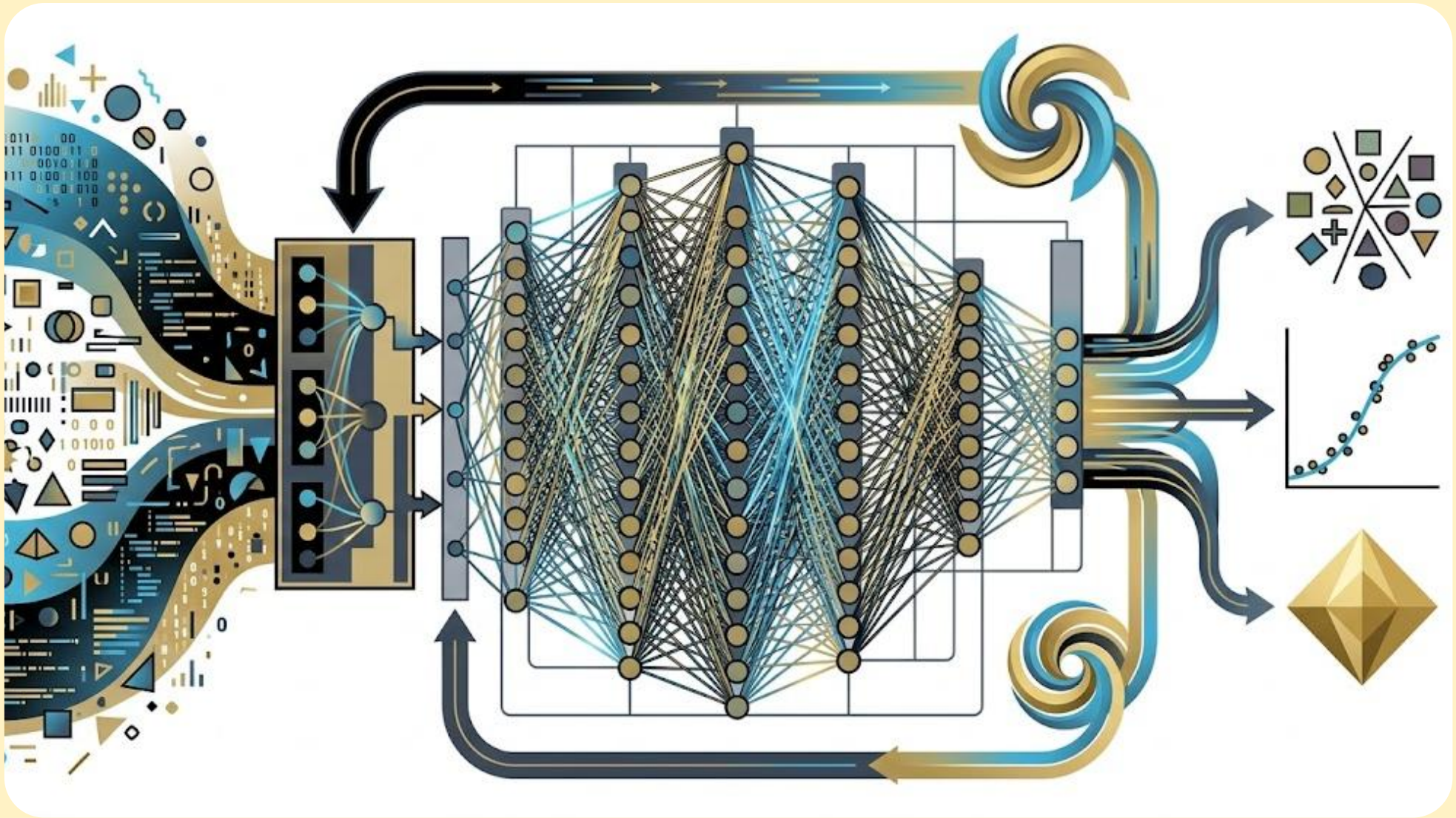
Coding, Systems, Agents

Machine Learning Model Lifecycle

Speaker:

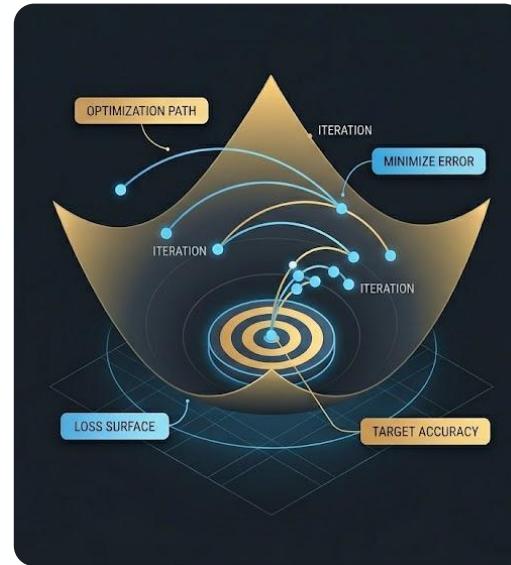
Suhas Dara (Master's Student, Purdue University)





Machine Learning Model Lifecycle

The Overarching Theme of Optimization



Algorithmic Optimization

- **Target:** Ground-truth training data
- **Error metric:** “Distance” from target
- **Optimization engine:** Iterative weight updates to minimize errors

Data Preparation

Raw Inputs

- **Many data sources, often unstructured**
 - Sensor readings
 - Logs
 - Databases
- **Data issues**
 - Missing values
 - Wrong formats
 - Outliers
 - Incompatible scales and units



The Preparation

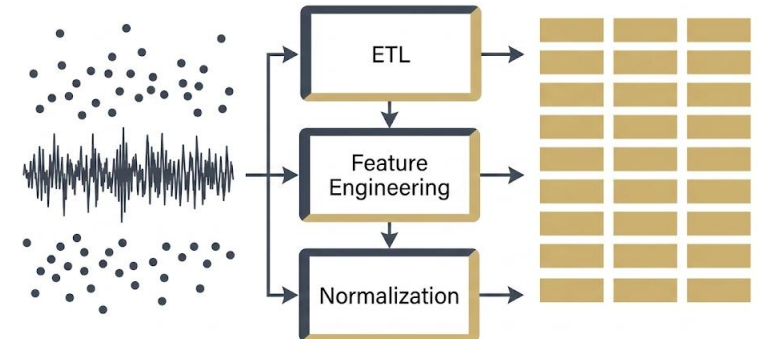
- **ETL & Cleaning:** Ingest and scrub raw inputs. Handle missing values, noise, and formatting issues
- **Feature Engineering:** Extract meaningful signals to decide what the model sees
- **Normalization/Scaling:** Put all features on an equal footing



Model-ready Features

Feature Vectors

Structured, normalized, and model-ready



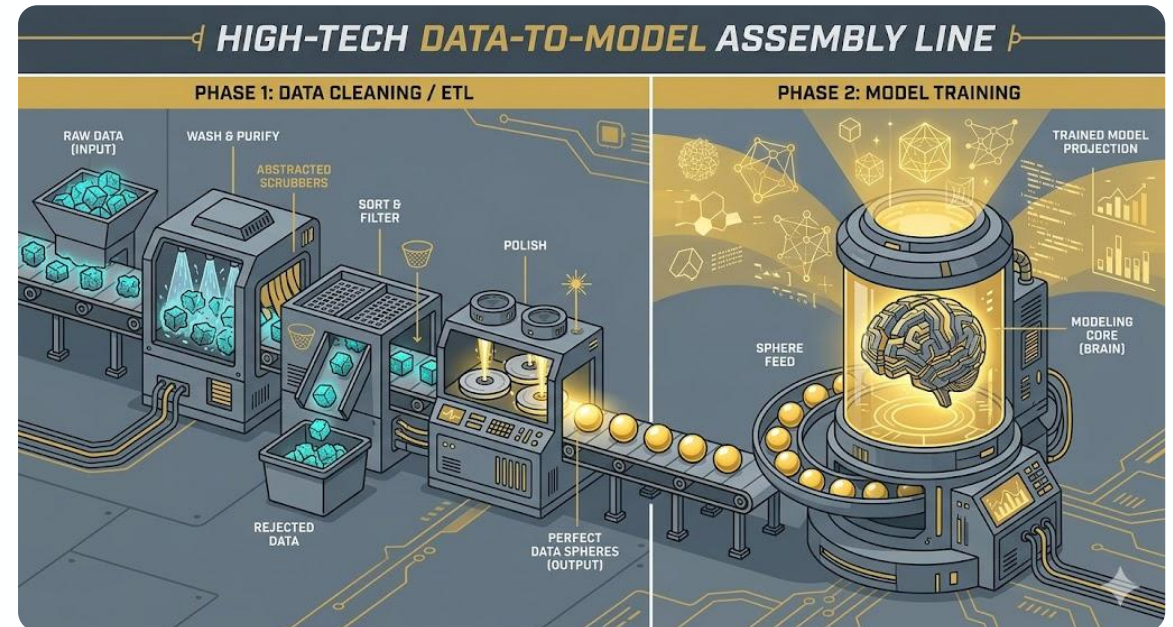
Upstream and Downstream

Foundational Data Preparation

- **ETL Pipelines:** Ingesting and scrubbing raw inputs
- **Feature Extraction:** Distilling signal from noise
- **Data Clean-up:** Ensuring learning stability

Intelligent Pattern Recognition

- **Model Learning:** Using cleaned data to update system weights
- **Evaluation:** Measuring true impact and iterating



The Complete ML Lifecycle

Before Training

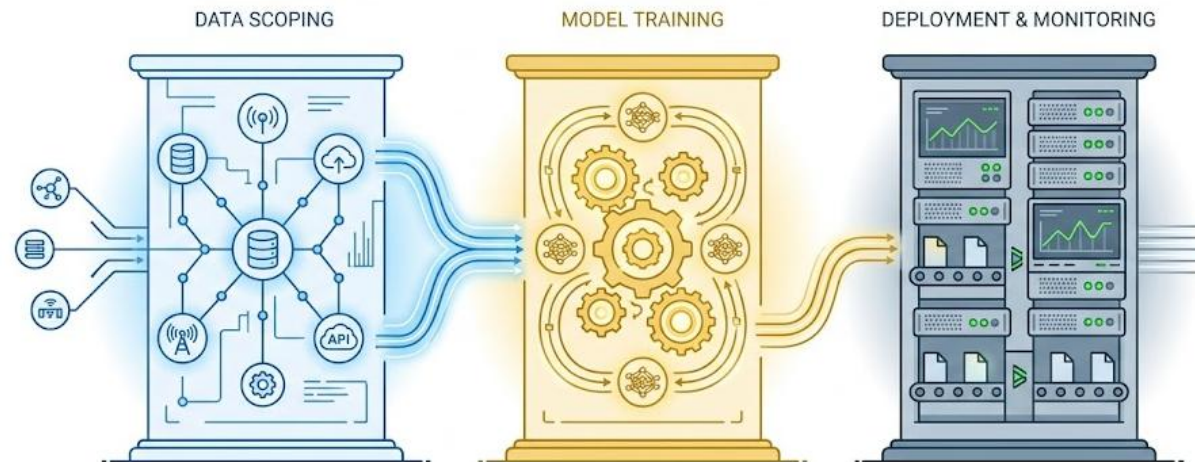
- Problem Definition
- Data Collection
- Preprocessing
- Feature Engineering

The Training Loop

- Model Selection
- Training: loss + optimizer
- Evaluation
- Hyperparameter Tuning

After Training

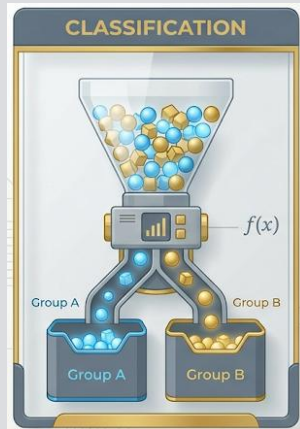
- Deployment
- Monitoring
- Retrain cycle



Goal-driven Model Selection

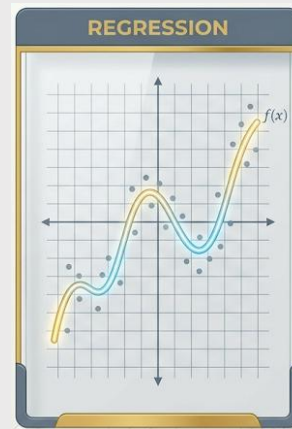
Classification

- **Goal:** Map inputs into discrete categories
- **Examples:**
 - Logistic Regression
 - Decision Trees
 - Deep Learning



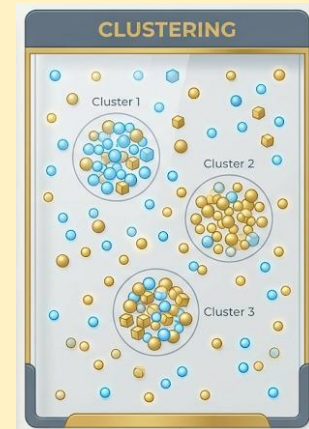
Regression

- **Goal:** Map inputs into continuous output space
- **Examples:**
 - Linear Models
 - Ridge/Lasso Regression
 - Deep Learning



Clustering

- **Goal:** Discover structural groupings
- **Examples:**
 - K-Means
 - DBSCAN
 - Deep Learning



Scale-driven Model Selection

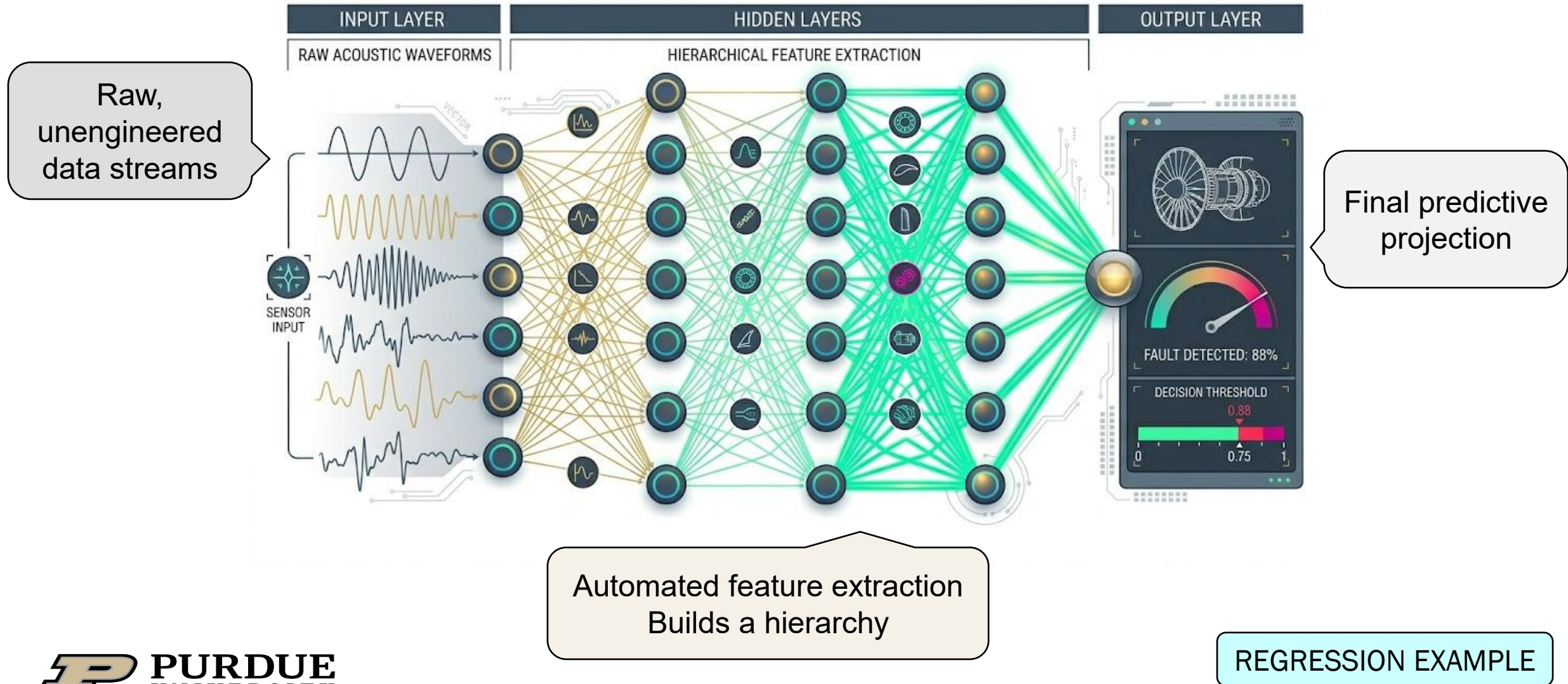
Classical / Low-Capacity Regime

- **Data Profile:** Small to medium unstructured datasets, heavy feature-engineering required
- **Advantage:** Highly interpretable, computationally lightweight, overfitting-resistant
- **Optimal Methods:**
 - Regularized Linear Regression
 - SVMs
 - Random Forests

Deep Learning / High-Capacity Regime

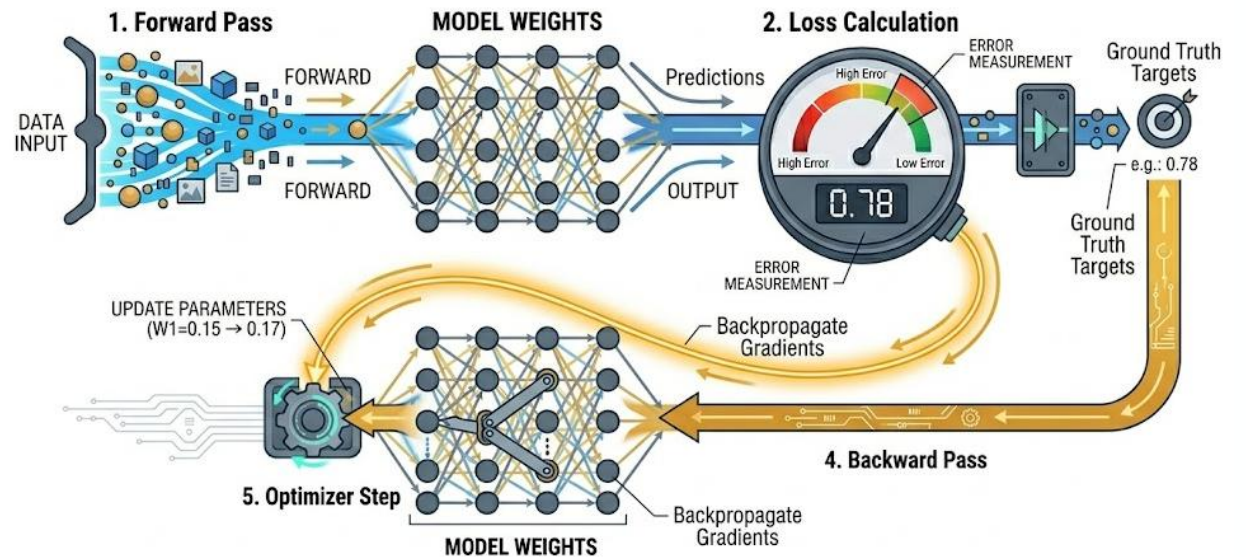
- **Data Profile:** Massive, high-dimensional scale; bypass manual feature-engineering as the model will learn itself
- **Advantage:** Scales with data volume, can capture complex, non-linear physical interactions
- **Optimal Methods:**
 - Multi-Layer Perceptrons
 - Convolutional Neural Networks
 - Transformers

Anatomy of Deep Learning



Anatomy of a Training Loop

```
for epoch in range(num_epochs):  
    # 1. Forward Pass: Generate predictions  
    predictions = model(X_train)  
  
    # 2. Calculate Loss: Measure error  
    loss = loss_function(predictions, y_train)  
  
    # 3. Zero Gradients: Clear memory  
    optimizer.zero_grad()  
  
    # 4. Backward Pass: Compute derivatives  
    loss.backward()  
  
    # 5. Optimizer Step: Update weights  
    optimizer.step()
```



Deconstructing Regression Losses

Mean Squared Error (MSE)

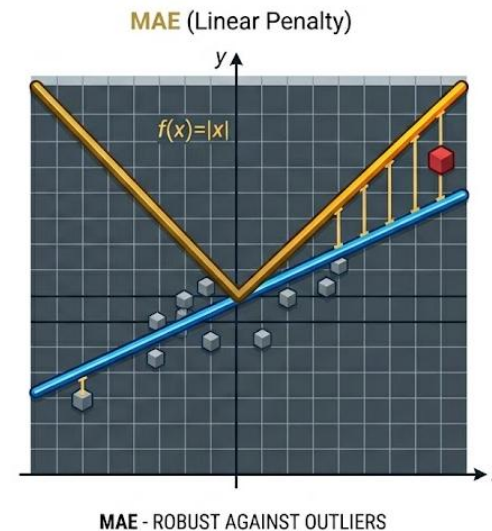
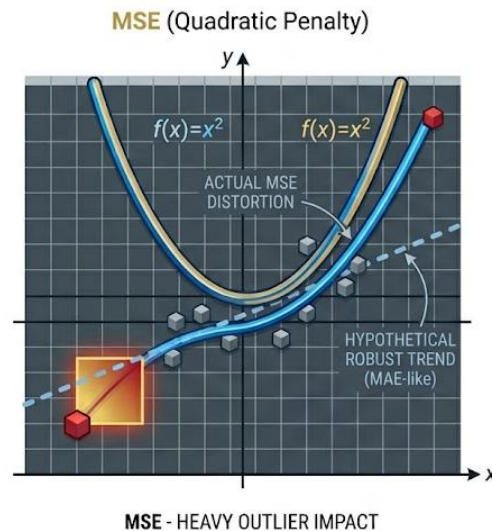
$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Errors are penalized quadratically. Lots of outliers could make it unstable, however, usually the logical choice.

Mean Absolute Error (MAE)

$$\mathcal{L}_{\text{MAE}} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Errors are penalized proportional to magnitude. Right choice when data contains extreme outliers or unpreventable noise.

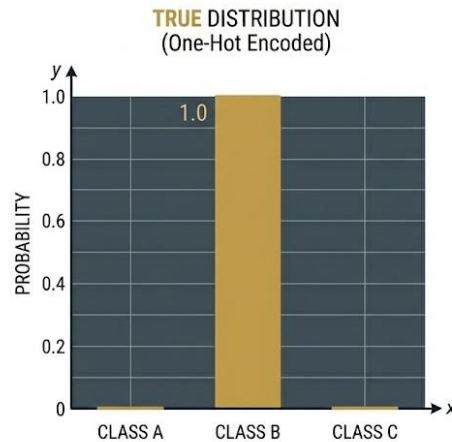


Deconstructing Classification Loss

True Distribution (y)

Absolute certainty represented via **One-Hot Encoding**.

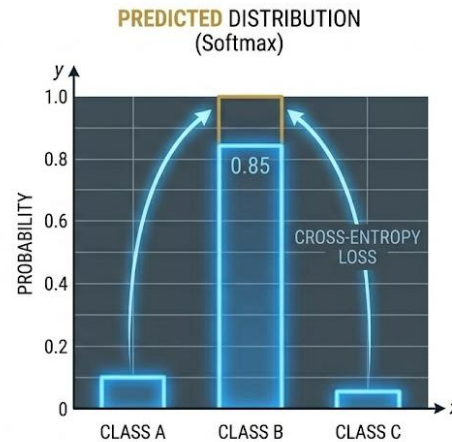
Example:
[0.0, 1.0, 0.0]



Predicted Distribution ($\hat{y}$)

Model outputs mapped to valid probabilities via **Softmax** or **Sigmoid**.

Example:
[0.1, 0.85, 0.05]



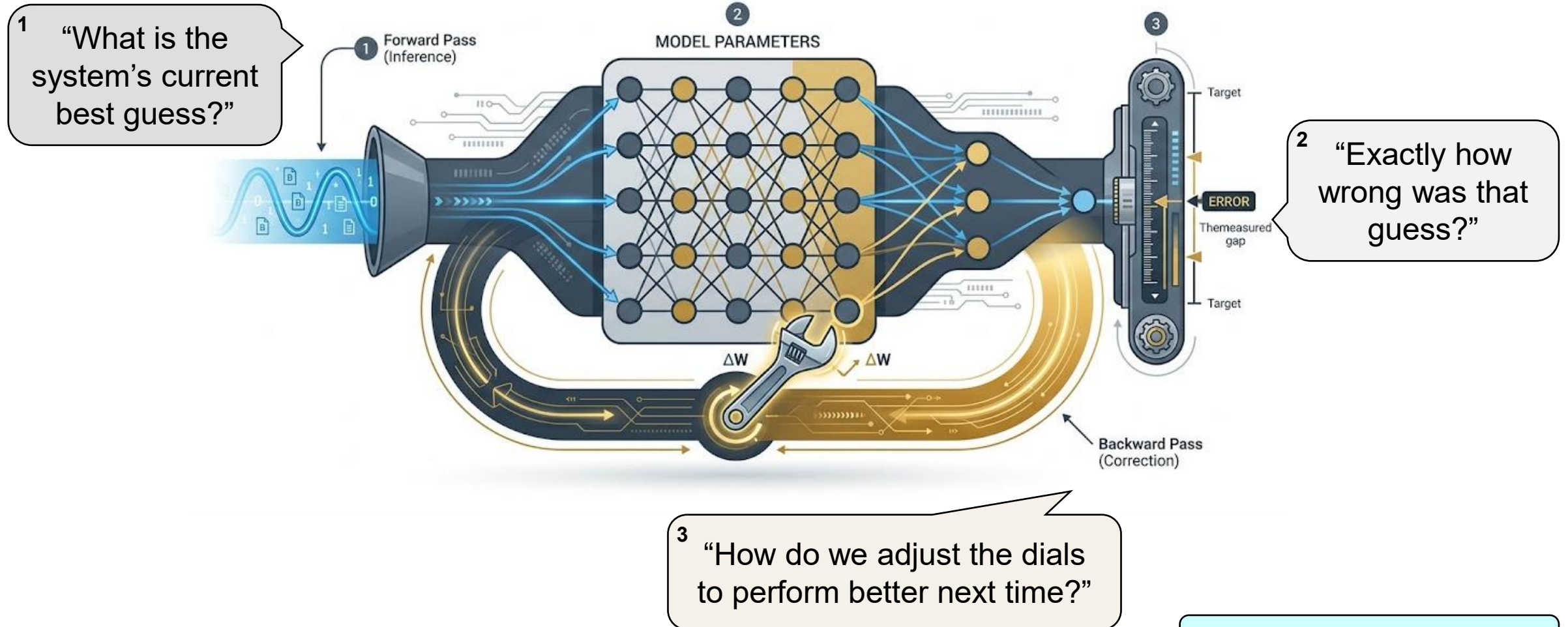
Cross-Entropy

Sums the log probabilities of the predicted outputs weighted strictly by the ground truth.

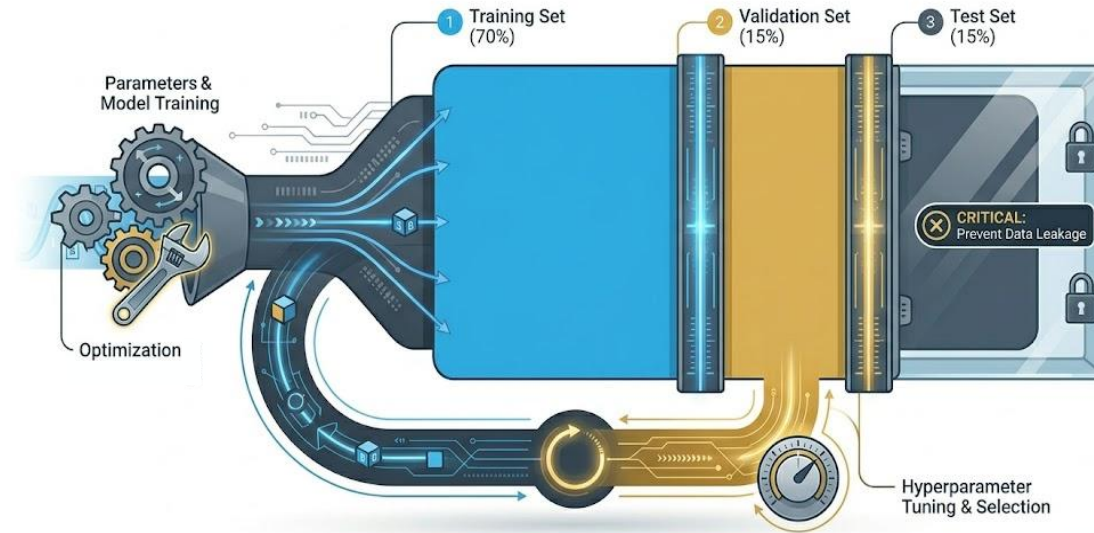
$$\mathcal{L}_{CE} = - \sum_{i=1}^c y_i \log(\hat{y}_i)$$

Asymmetric logarithmic penalty when the model assigns a probability near zero to the true class.

Completing the Cycle



Splitting the Dataset *(How not to Cheat)*



Training Set (~70%)

Active learning environment

Model parameters updated based on this data.

Validation Set (~15%)

Iterative tuning environment

Model hyperparameters updated based on this data.

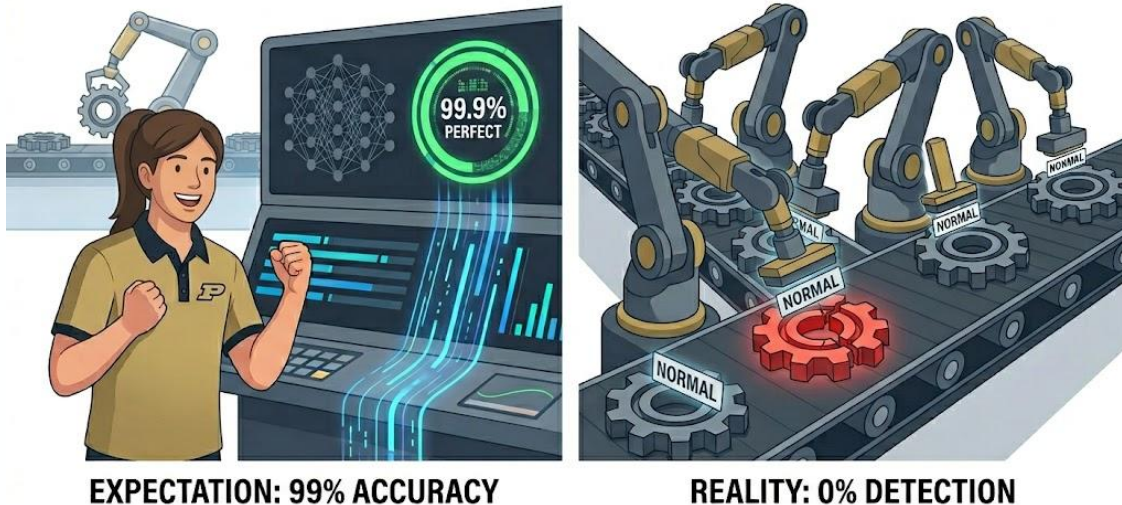
Test Set (~15%)

Evaluation standard

Unseen data until the final model is decided.

The Accuracy Trap

ML TRAINING: EXPECTATION VS. REALITY



The Illusion

Overall accuracy measures total percentage of correct predictions:

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Samples}}$$

99.9% accuracy → “Model is performing great”

The Reality

Model is lazy: Simply predict the easy thing
Bottom line: We accomplish nothing!

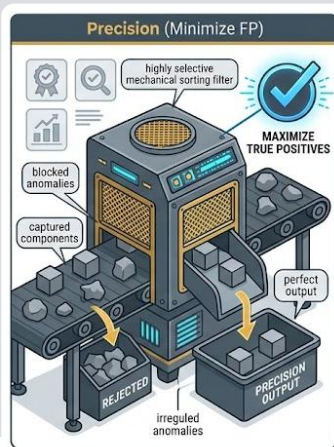
Beyond Accuracy

Precision

Goal: Maximize quality of “positive” predictions

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

TP: True Positive
FP: False Positive

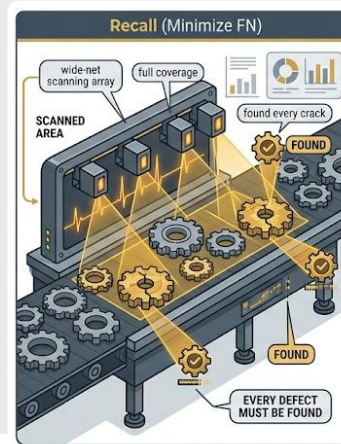


Recall

Goal: Maximize quantity of actual anomalies detected

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

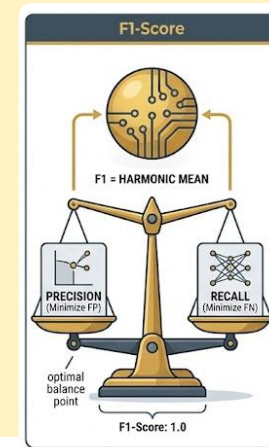
TP: True Positive
FN: False Negative



F1-Score

Goal: Harmonic mean balancing precision and recall

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$



More Precision vs Recall Examples

<u>Use Case</u>	<u>Prefer</u>	<u>Why</u>	<u>False +ve Cost</u>	<u>False -ve Cost</u>
Acoustic anomaly detection in an Automated Assembly Line	Precision	Halting high-speed production is extremely expensive, so alerts must be absolutely trustworthy.	Critical Financial Waste: Unnecessary line shutdown wasting thousands of dollars per minute in lost factory throughput.	Low Operational Impact: A defective component slips through to the next station, handled by secondary checks.
Micro-defect scanning in an Aerospace Turbine Production Line	Recall	Safety-critical component. Absolute zero tolerance for missing active internal structural flaws.	Low Operational Impact: A healthy blade is flagged for secondary manual inspection by a technician.	Catastrophic Failure: Uncaught internal micro-crack leads to mid-flight turbine disintegration and total system loss.
Autonomous Vehicle Emergency Braking	Recall	Failing to detect a true physical obstacle results in severe, irreversible real-world collisions.	Moderate Friction: Vehicle brakes unnecessarily for floating debris or heavy steam, causing minor passenger discomfort.	Fatal Safety Impact: System fails to engage brakes for a pedestrian or vehicle, causing direct high-speed impact.

The Complete ML Lifecycle

Underfitting (High Bias)

- **Lacks capacity** to capture underlying patterns
- Focuses on capturing the **mean (bias)** of the distribution

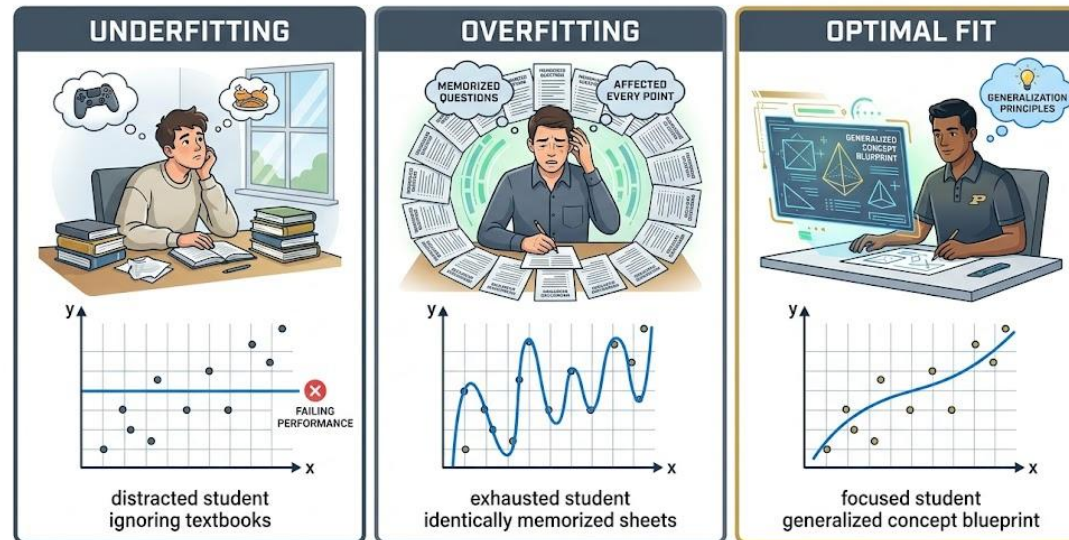
Overfitting (High Variance)

- **Memorizes** training noise and fails on unseen distributions
- Focuses on capturing the **variance** of the distribution

Optimal fit (The Goal)

- Captures **true underlying signal**
- **Balance** between bias and variance

MACHINE LEARNING FIT: A STUDY HABITS ANALOGY



Regularization *(Chasing the Optimal Fit)*

L1 Regularization (Lasso)

Adds absolute value of parameters as a penalty to the loss → avoid large parameters

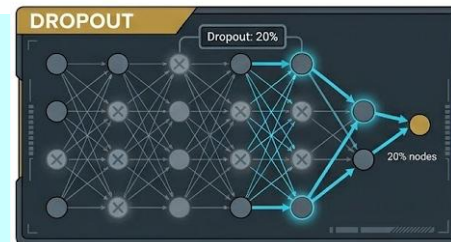


L2 Regularization (Ridge)

Adds squared value of parameters as a penalty to the loss → avoid large parameters



Randomly deactivates a percentage of neurons in hidden layers loss → avoid over-reliance on specific neurons



Dropout (in DL)



Halts the training loop when the validation loss plateaus or starts to worsen

Early Stopping

Robust Validation & Systematic Tuning

Cross Validation

- **Shuffle** all non-test data into **K folds** and take turns to leave one fold from training each time
- **Evaluate** as an average of **using every fold as the validation data**
- **Stratification**: Prevent folds from accidentally not having anomalous data due to class imbalances

Hyperparameter Tuning

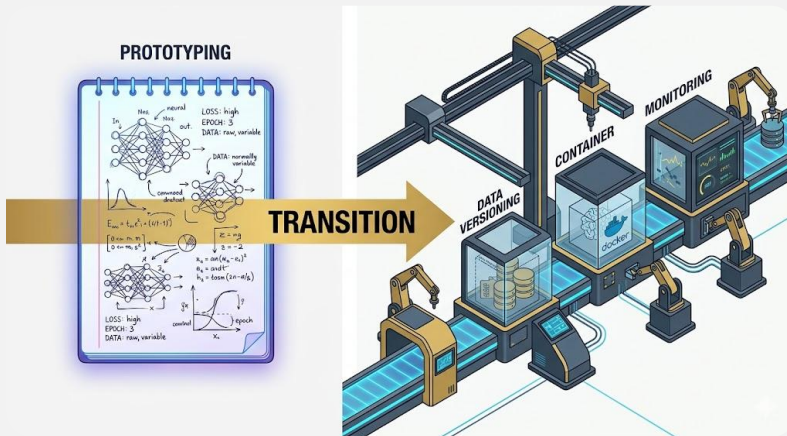
- **Adjusting manual** external “knobs” of the model before training begins
- Use **K-Fold Cross Validation** for scoring each configuration
- **EXPONENTIAL PROBLEM!**
- **Grid search**: Manually test a subset of all combinations using reasonable values



Prototyping → Production

The Notebook (Discovery)

- **Interactive Lab Bench** with ad-hoc and manual execution
- **Risk:** “It works on my machine”
- **Goal:** Identify what works



The Pipeline (Delivery)

- **Automated** with deterministic and modular workflows
- **Risk:** Systematic data/model drift over time
- **Goal:** Reliable and Reproducible performance at scale

- ✓ **Versioning**
- ✓ **Containerization**
- ✓ **Monitoring**

Just some resources

- **Kaggle:** Standard for free, open-source datasets
- **Hugging Face:** Standard for pre-trained models to build projects
- **YouTube:** There are many good quality channels. The following are what I personally like
 - **3Blue1Brown**
 - **StatQuest with Josh Starmer**
- **One-off explanation websites:** Highly interactive browser-based sandboxes for building and understanding neural networks
 - **CNN Explainer**
 - **TensorFlow Playground**
- Cool blog on using LLMs effectively: <https://sureprompts.com/blog/ai-prompt-cheat-sheet>

Thank You

